❖ Arrays

Q. What is an Array?

Ans. An **array** is a collection of elements of the **same data type** stored in **contiguous memory locations**. Each element is accessed using its **index**, starting from **0**.

Advantages

- Stores multiple values in one variable.
- Fast access using index.
- Easy to traverse using loops.
- Reduces code redundancy.

Disadvantages

- Fixed size.
- Can store only one type of data.
- Insertion and deletion are costly.

Syntax

Declaration:

```
dataType[] arrayName;
```

Example: int[] marks;

Memory Allocation

```
marks = new int[5];    or

int[] marks = new int[5];
```

Initialization:

```
int[] marks = {78, 85, 90, 65, 88};
```

Accessing Elements

```
System.out.println(marks[0]);
System.out.println(marks[3]);
```

Output

```
78
65
```

Updating Elements

```
marks[2] = 95;
```

Example Program

```
public class ArrayExample {
    public static void main(String[] args) {
        int[] marks = {80, 75, 90, 88, 95};
        for(int i = 0; i < marks.length; i++) {
            System.out.println(marks[i]);
        }
    }
}
```

Output

```
80
75
90
88
95
```

❖ **One-Dimensional Arrays**

➢ A **One-Dimensional Array (1D Array)** stores data in a single row.

Example Representation

Index : 0 1 2 3 4
Value : 12 25 18 40 50

Declaration

```
int[] numbers = new int[5];
```

Initialization

```
int[] numbers = {10,20,30,40,50};
```

❖ **Traversing Array**

Using for loop

```
for(int i=0;i<numbers.length;i++){
    System.out.println(numbers[i]);
}
```

Using enhanced for loop

```
for(int num : numbers){
        System.out.println(num);
}
```

Example

```
public class OneDArray {
        public static void main(String[] args) {
                int[] arr = {5,10,15,20,25};
                for(int value : arr){
                        System.out.println(value);
                }
         }
}
```

Applications

- Student marks
- Employee IDs
- Monthly sales
- Temperatures
- Bank transactions

3. Multi-Dimensional Arrays

➢ A **Multi-Dimensional Array** is an array containing one or more arrays.

Most common: **Two-Dimensional Array (2D Array)**

It stores data in rows and columns.

Example

```
1 2 3
4 5 6
7 8 9
```

Syntax

```
int[][] matrix = new int[3][3];
```

Initialization

```
int[][] matrix = {
                {1,2,3},
                {4,5,6},
                {7,8,9}
        };
```

Accessing Elements

```
System.out.println(matrix[0][0]);
System.out.println(matrix[2][1]);
```

Output

```
1
8
```

Traversing 2D Array

```
for(int i=0;i<matrix.length;i++){
    for(int j=0;j<matrix[i].length;j++){
        System.out.print(matrix[i][j]+" ");
    }
    System.out.println();
}
```

Output

```
1 2 3
4 5 6
7 8 9
```

Example :

```
public class MatrixExample {
    public static void main(String[] args) {
        int[][] marks = {
                {80,90},
                    {70,85},
                {95,88}
        };
        for(int i=0;i<marks.length;i++){
            for(int j=0;j<marks[i].length;j++){
                System.out.print(marks[i][j]+" ");
                 }
                System.out.println();
        }
    }
}
```

Applications

- Matrix operations
- Image processing
- Seating arrangement
- Chess board
- Game development

❖ Array Operations

1. Traversing

- Reading every element one by one.

```
for(int i=0;i<arr.length;i++){
        System.out.println(arr[i]);
}
```

✓ Time Complexity: **O(n)**

2. Insertion

- Arrays have fixed size.

Example

```
int[] arr = new int[6];

arr[0]=10;
arr[1]=20;
arr[2]=30;
arr[3]=40;
```

To insert in between, shift elements.

Example:

Before: 10 20 30 40
Insert 25
After: 10 20 25 30 40

✓ Time Complexity: **O(n)**

3. Deletion

- Delete an element by shifting remaining elements.

Example

Before: 10 20 30 40 50
Delete 30
After: 10 20 40 50

✓ Time Complexity: **O(n)**

4. Searching

Linear Search

- Checks each element one by one.

```
int key = 30;

for(int i=0;i<arr.length;i++){
    if(arr[i]==key){
        System.out.println("Found");
    }
}
```

✓ Time Complexity: **O(n)**

❖ **Binary Search**

- Works only on sorted arrays.

Example

10 20 30 40 50

✓ Time Complexity: O(log n)

5. Updating

- Replace an existing value.

arr[2]=100;

Before: 10 20 30 40
After: 10 20 100 40

✓ Time Complexity: O(1)

6. Sorting

- Sorting arranges elements in ascending or descending order.

Example

Before: 40 20 50 10 30
After: 10 20 30 40 50

Using Java

```
import java.util.Arrays;
Arrays.sort(arr);
```

✓ Time Complexity: O(n log n)

Strings in Java

Q.What is a String?

Ans. A **String** is a sequence of characters used to store text. In Java, strings are objects of the String class and are immutable (cannot be changed after creation).

Declaration

```
String name = "Prince";
```

Creating Strings

```
String s1 = "Hello";
String s2 = new String("World");
```

Characteristics of String

- Immutable (cannot be changed after creation)
- Stored as objects
- Supports many built-in methods
- Widely used for text processing

Creating Strings

Method 1: Using String Literal

```
String s1 = "Java";
```

Method 2: Using new Keyword

```
String s2 = new String("Programming");
```

❖ **String Memory (String Pool)**

Java stores string literals in a special memory area called the **String Constant Pool**.

Example:

```
String s1 = "Java";
String s2 = "Java";
```

- Both s1 and s2 refer to the same object.
- Using new creates a separate object.

```
String s3 = new String("Java");
```

Common String Methods

Method	Description	Example
length()	Returns string length	"Java".length()
charAt()	Returns character at index	"Java".charAt(1)
substring()	Extracts part of string	"Programming".substring(0,7)
equals()	Compares values	"Java".equals("Java")
equalsIgnoreCase()	Ignores case while comparing	"java".equalsIgnoreCase("JAVA")
toUpperCase()	Converts to uppercase	"java"
toLowerCase()	Converts to lowercase	"JAVA"
trim()	Removes extra spaces	" Hello "
replace()	Replaces characters	"Java".replace('a','o')
contains()	Checks substring	"Java Programming"
startsWith()	Checks prefix	"Java".startsWith("Ja")
endsWith()	Checks suffix	"Java".endsWith("va")
indexOf()	Returns index	"Java".indexOf('v')

Example:

```
public class StringExample {
        public static void main(String[] args) {

                String name = "Java Programming";

                System.out.println("Length: " + name.length());
                System.out.println("Uppercase: " + name.toUpperCase());
                System.out.println("Lowercase: " + name.toLowerCase());
                System.out.println("Substring: " + name.substring(0,4));
                System.out.println("Contains Java: " + name.contains("Java"));
        }
}
```

Key Differences: Array vs String

Array	String
Stores elements of the same data type	Stores characters (text)
Fixed size	Immutable in Java
Access using index	Access using methods and index
Example: int[] arr = {1,2,3};	Example: String s = "Java";

❖ **StringBuffer**

- StringBuffer is a mutable class used to modify strings without creating new objects.
- **Mutable** means the content can be changed.

Package

java.lang.StringBuffer

Q. Why StringBuffer?

Ans. Instead of creating a new object every time, the existing object is modified.

Creating StringBuffer

```
StringBuffer sb = new StringBuffer("Java");
```

Common Methods

Method	Description
append()	Adds text
insert()	Inserts text
delete()	Deletes characters
replace()	Replaces characters
reverse()	Reverses string
capacity()	Returns capacity
length()	Returns length
charAt()	Returns character

Example

```
StringBuffer sb = new StringBuffer("Java");
sb.append(" Programming");
System.out.println(sb);
```

Output: Java Programming

Example

```
StringBuffer sb = new StringBuffer("Hello");
sb.reverse();
System.out.println(sb);
```

Output: olleH

Advantages

- Fast
- Memory efficient
- Thread-safe (synchronized)
- Suitable for multi-threaded applications

❖ **StringBuilder**

- StringBuilder is similar to StringBuffer, but it is **not synchronized**.
- It is faster than StringBuffer.

Creating Object

```
StringBuilder sb = new StringBuilder("Java");
```

Example

```
StringBuilder sb = new StringBuilder("Programming");
sb.append(" Language");
System.out.println(sb);
```

Output: Programming Language

Reverse Example

```
StringBuilder sb = new StringBuilder("Java");

System.out.println(sb.reverse());
```

Output: avaJ

Advantages

- Faster than StringBuffer
- Less memory usage
- Best for single-thread applications

4. String Comparison

Java provides multiple ways to compare strings.

- **Using == Operator**

  Compares object references.

  ```
  String s1 = "Java";
  String s2 = "Java";
  System.out.println(s1 == s2);
  ```

Output: true

- **Using equals()**

  Compares actual values.

  ```
  String s1 = new String("Java");
  String s2 = new String("Java");
  System.out.println(s1.equals(s2));
  ```

Output: true

- **Using equalsIgnoreCase()**

  ```
  String s1 = "JAVA";
  String s2 = "java";
  System.out.println(s1.equalsIgnoreCase(s2));
  ```

Output: true

- **Using compareTo()**

Returns

- 0 → Equal
- Positive → First string is greater
- Negative → Second string is greater

Example

```
String s1="Apple";
String s2="Banana";
System.out.println(s1.compareTo(s2));
```

Output: Negative Value

❖ **Difference Between == and equals()**

==	**equals()**
Compares references	Compares values
Faster	Slightly slower
Used for object reference	Used for string content

5. String Manipulation Methods

String manipulation means performing different operations on strings.

1. length()

```
String s="Programming";
System.out.println(s.length());
```

Output: 11

2. charAt()

```
System.out.println(s.charAt(3));
```

Output: g

3. substring()

```
System.out.println(s.substring(0,7));
```

Output: Program

4. concat()

```
String first="Java";
String second=" Programming";
System.out.println(first.concat(second));
```

Output: Java Programming

5. replace()

```
String s="Java";
System.out.println(s.replace('a','o'));
```

Output: Jovo

6. toUpperCase()

```
System.out.println(s.toUpperCase());
```

Output: JAVA

7. toLowerCase()

```
System.out.println(s.toLowerCase());
```

Output: java

8. trim()

```
String s=" Hello ";
System.out.println(s.trim());
```

Output: Hello

9. contains()

```
String s="Java Programming";
System.out.println(s.contains("Java"));
```

Output: true

10. split()

```
String s="Java Python C++";
String[] arr=s.split(" ");
for(String word:arr)
        System.out.println(word);
```

Output

Java
Python
C++

11. startsWith()

```
System.out.println("Java".startsWith("Ja"));
```

Output: true

12. endsWith()

```
System.out.println("Java".endsWith("va"));
```

Output: true

13. indexOf()

```
System.out.println("Programming".indexOf('g'));
```

Output: 3

Important Questions

1. Explain One-Dimensional Arrays in Java with a suitable example.
2. Explain Multi-Dimensional Arrays in Java with a suitable example.
3. Explain various array operations in Java with suitable examples.
4. Explain the String class and its commonly used methods with suitable examples.
5. Differentiate between String, StringBuffer, and StringBuilder.
6. 6. Explain String comparison methods (==, equals(), and compareTo()).
7. Explain any six String manipulation methods with suitable examples.
8. Explain String immutability and its advantages.
9. Differentiate between the == operator and the equals() method in Java.
10. Write Java programs for matrix operations using two-dimensional arrays.

*****